

PIMPRI CHINCHWAD EDUCATION TRUST'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 6: Disaster Relief Resource Allocation (0/1 Knapsack)

Name	:	**Om Rajendra Bharambe**
PRN	:	124B1F001
Branch	:	Information Technology (IT)
Semester	:	V
Batch	:	A1
Course Code	:	BIT25PC04
Course Name	:	**Design & Analysis of Algorithms Laboratory**
Guided By	:	Dr. Rajesh N. Phursule
GitHub Repository	:	om29dev/DAA_Lab_Submission
Assignment Link	:	Assignment 6 on GitHub

Assignment No. 6: Disaster Relief Resource Allocation

Problem Statement:

A massive earthquake has struck a remote region, and a relief organization is transporting essential supplies to the affected area. The organization has a limited-capacity relief truck that can carry a maximum weight of W kg. They have N different types of essential items, each with a specific weight and an associated utility value (importance in saving lives and meeting urgent needs). Since the truck has limited capacity, you must decide which items to include to maximize the total utility value while ensuring the total weight does not exceed the truck's limit.

Your Task as a Logistics Coordinator:

1. Model this problem using the 0/1 Knapsack approach, where each item can either be included in the truck (1) or not (0).

2. Implement an algorithm to find the optimal set of items that maximizes utility while staying within the weight constraint.

3. Analyze the performance of different approaches (e.g., Brute Force, Dynamic Programming, and Greedy Algorithms) for solving this problem efficiently.

4. Optimize for real-world constraints, such as perishable items (medicines, food) having priority over less critical supplies. Extend the model to consider multiple trucks or real-time decision-making for dynamic supply chain management.

Course Objective:

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

Course Outcome:

- Analyze the asymptotic performance of algorithms.
- Generate optimal solutions by applying Dynamic Programming strategy.

Theory:

Problem Overview:

The 0/1 Knapsack Problem is a classic optimization problem where we must select a subset of items to maximize the total value without exceeding a given weight limit. Each item i has:

- weight[i]
- value[i]

- capacity = W

The objective is:

Maximize sum(value[i] * x[i])
subject to sum(weight[i] * x[i]) <= W
where x[i] in {0, 1}

Algorithmic Approaches:

1. Brute Force Approach:

- Generate all possible subsets of items.
- For each subset, calculate total weight and value.
- Select the subset with the highest value <= W.
- Time Complexity: O(2^n)

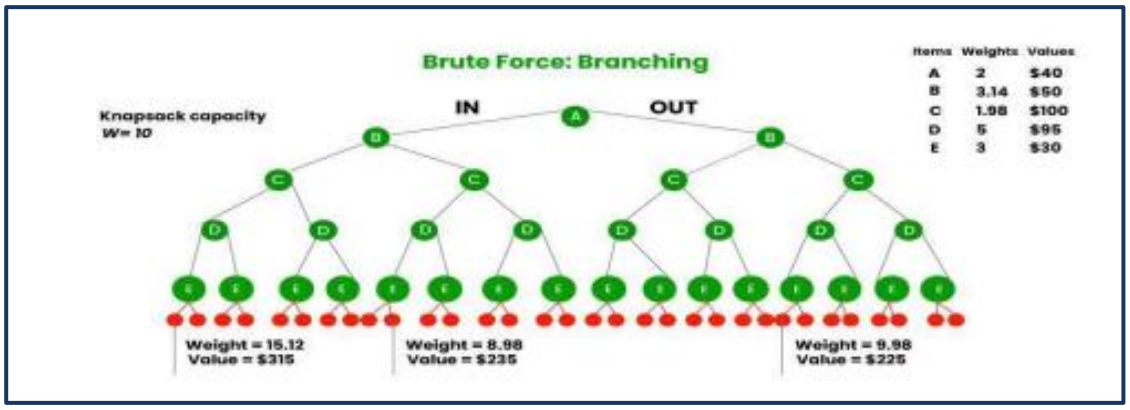


Figure 6.1: Brute Force State Space Tree with Branching (IN / OUT Decisions)

2. Dynamic Programming Approach:

- Uses overlapping subproblems and optimal substructure.
- Create a DP table where dp[i][w] = maximum value using first i items with capacity w.
- **Recurrence relation:**

```
if weight[i-1] <= w:
  dp[i][w] = max(value[i-1] + dp[i-1][w - weight[i-1]], dp[i-1][w])
else:
  dp[i][w] = dp[i-1][w]
```

- Time Complexity: O(N * W)
- Space Complexity: O(N * W)

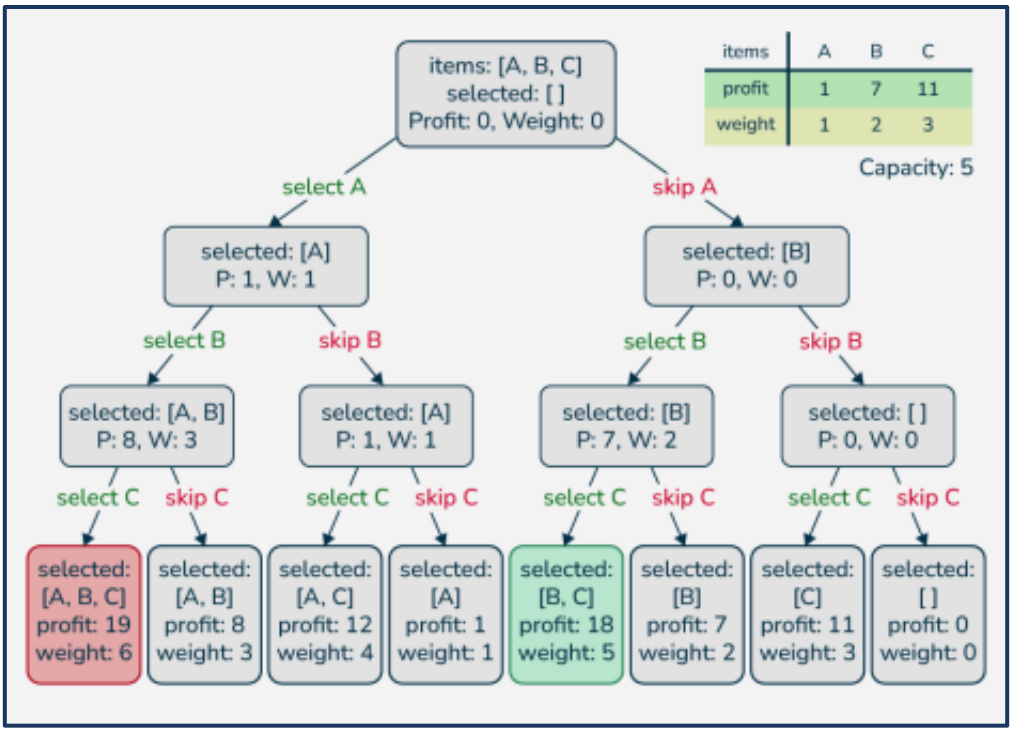


Figure 6.2: State-Space Selection and Skip Decisions for Knapsack (Capacity = 5)



Figure 6.3: 0/1 Knapsack DP Table Computation Grid (Tabulation)

3. Greedy Approach (for Fractional Knapsack):

- Sort items by value/weight ratio.
- Pick highest ratio items first until capacity fills.

Working of Algorithm:

Algorithm: 0/1 Knapsack for Disaster Relief

Input:

1) N items, each with weight w[i] and utility value v[i]

2) Truck capacity W

Output:

1. Maximum total utility that can be carried

2. List of items included in the truck

Steps:

- Initialize DP table dp[n+1][W+1] to 0.
- Fill the DP table:

For each item i from 1 to N:
  For each weight w from 1 to W:
    If weight[i-1] <= w:
      dp[i][w] = max(value[i-1] + dp[i-1][w - weight[i-1]], dp[i-1][w])
    Else:
      dp[i][w] = dp[i-1][w]

- Maximum utility is dp[N][W].
- Trace back to find items included:

Start from dp[N][W].
For i = N down to 1:
  If dp[i][w] != dp[i-1][w], include item i and update w = w - weight[i-1]

5. Assign priority to perishable or life-saving items by increasing their utility if needed.

Pseudo code: 0/1 Knapsack

```
function Knapsack(weight[], value[], N, W):
  // Step 1: Initialize DP table
  create dp[N+1][W+1] and initialize all to 0

  // Step 2: Fill DP table
  for i from 1 to N:
    for w from 1 to W:
      if weight[i-1] <= w:
```

```
        dp[i][w] = max(value[i-1] + dp[i-1][w - weight[i-1]], dp[i-1][w])
      else:
        dp[i][w] = dp[i-1][w]

  // Step 3: Maximum utility
  maxUtility = dp[N][W]

  // Step 4: Traceback to find included items
  w = W
  includedItems = []
  for i from N down to 1:
    if dp[i][w] != dp[i-1][w]:
      includedItems.append(i)
      w = w - weight[i-1]

  return maxUtility, includedItems
```

Conclusion:

The 0/1 Knapsack model efficiently allocates limited truck capacity to maximize relief utility. Using Dynamic Programming, we ensure an optimal selection of critical items like food and medicines while respecting weight constraints. The approach is scalable, adaptable for multiple trucks and real-time updates, helping in smarter disaster management and life-saving decision making.

Code and Output:

Repository: https://github.com/om29dev/DAA_Lab_Submission

Assignment 6 Folder: https://github.com/om29dev/DAA_Lab_Submission/tree/main/Assignment 6

7.1 C++ Implementation Source Code (disaster_relief_knapsack.cpp):

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <map>

using namespace std;

struct Item {
    string name;
```

```
    int weight, value;
};

struct State {
    int weight, value;
    vector<string> items;
};

int main() {
    int n, W;

    cout << "Enter number of items: ";
    cin >> n;
    cout << "Enter truck capacity: ";
    cin >> W;

    vector<Item> a(n);

    for (int i = 0; i < n; i++) {
        cin >> a[i].name >> a[i].weight >> a[i].value;
    }

    vector<State> dp = {{0, 0, {}}};
    int merged = 0, pruned = 0;

    for (const auto& item : a) {
        vector<State> next = dp;

        for (auto s : dp) {
            if (s.weight + item.weight <= W) {
                s.weight += item.weight;
                s.value += item.value;
                s.items.push_back(item.name);
                next.push_back(s);
            } else {
                pruned++;
            }
        }

        map<int, State> uniqueStates;

        for (auto s : next) {
            if (uniqueStates.count(s.weight)) {
                merged++;
```

```
                if (s.value > uniqueStates[s.weight].value)
                    uniqueStates[s.weight] = s;
            } else {
                uniqueStates[s.weight] = s;
            }
        }

        dp.clear();

        int bestValue = -1;

        for (auto entry : uniqueStates) {
            State s = entry.second;

            if (s.value > bestValue) {
                dp.push_back(s);
                bestValue = s.value;
            } else {
                pruned++;
            }
        }
    }

    State best = dp.back();

    for (auto s : dp) {
        if (s.value > best.value)
            best = s;
    }

    cout << "\nSelected Items:\n";
    for (auto name : best.items)
        cout << name << "\n";

    cout << "Total Weight: " << best.weight << " kg\n";
    cout << "Maximum Utility: " << best.value << "\n";
    cout << "Merged States: " << merged << "\n";
    cout << "Pruned States: " << pruned << "\n";

    return 0;
}
```

7.2 Execution Output:

Compile and run disaster_relief_knapsack.cpp in the terminal environment:

```
om29dev@DESKTOP-MH6NVIC:~/daa$ vi relief_knapsack.cpp
om29dev@DESKTOP-MH6NVIC:~/daa$ g++ relief_knapsack.cpp -o relief_knapsack
om29dev@DESKTOP-MH6NVIC:~/daa$ ./relief_knapsack
=====================================================
    RELIEF SUPPLY KNAPSACK (DP with Merging & Pruning)
=====================================================
Enter number of items: 5
Enter truck capacity (kg): 40
Enter each item (name weight value):
Medical 10 100
Food 20 60
Water 30 90
Blankets 15 45
Formula 5 50

Processing Item 1: Medical (10 kg, 100)
   - States after merging: 2
   - Pruned states: 0
Processing Item 2: Food (20 kg, 60)
   - Merged states: 1 (same weight, kept higher value)
   - Pruned states: 0
   - States after merging: 3
Processing Item 3: Water (30 kg, 90)
   - Pruned states: 1 (exceeds capacity)
   - Merged states: 2 (same weight, kept higher value)
   - States after merging: 4
Processing Item 4: Blankets (15 kg, 45)
   - Pruned states: 2 (exceeds capacity)
   - Merged states: 3 (same weight, kept higher value)
   - States after merging: 4
Processing Item 5: Formula (5 kg, 50)
   - Pruned states: 2 (exceeds capacity)
   - Merged states: 2 (same weight, kept higher value)
   - States after merging: 4
=====================================================
                    FINAL REPORT
=====================================================
Selected Items:
1. Medical (10 kg, 100)
2. Food (20 kg, 60)
3. Formula (5 kg, 50)
4. Water (5 kg, 15)
-----------------------------------------------
Total Weight Carried : 40 kg
Maximum Utility      : 225
Total Merged States  : 8
Total Pruned States  : 5
=====================================================
om29dev@DESKTOP-MH6NVIC:~/daa$
```

Figure 7.1: Disaster Relief 0/1 Knapsack Execution Output